Chapter 4
File Systems

- Files
- Directories
- File System Implementation
- Example File Systems

File System Implementation

- Contents
  - File System Layout
  - Implementing Files
  - Implementing Directory
  - Shared Files
  - Disk Space Management
  - File System Reliability
  - File System Performance

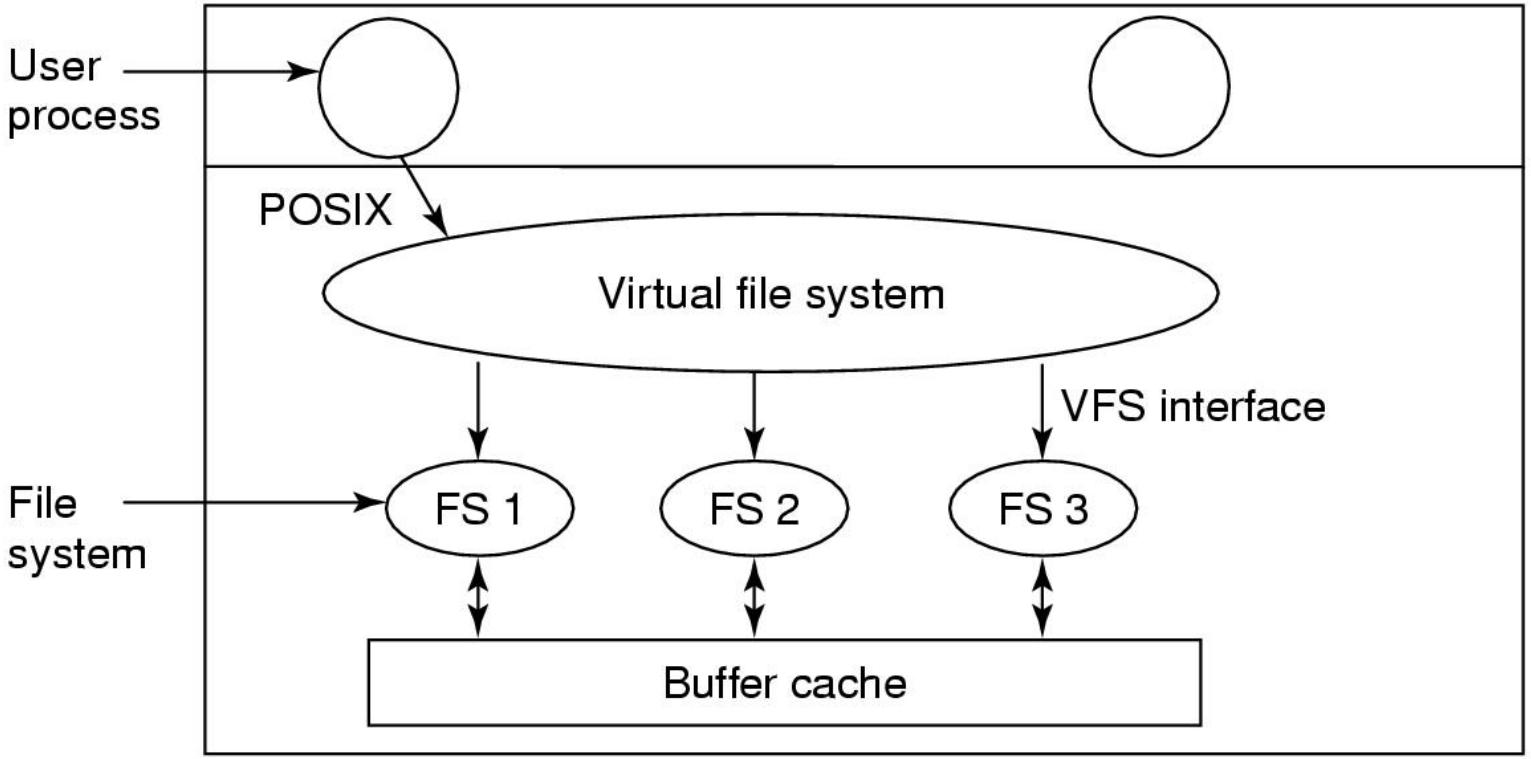
Journaling File Systems (1)

- Journaling File System
  - Keep a log of what the file system is going to do before it does it
  - If the system crashes before it completes its planned work
    - Rebooting the system
    - Look in the log to see what's going on at the time of crash
    - Finish the job
  - E.g., NTFS, ext3, ReiserFS

Journaling File Systems (2)

- Operations required to **remove a file** in UNIX
  - Remove the file from its directory.
  - Release the i-node to the pool of free i-nodes.
  - Return all the disk blocks to the pool of free disk blocks.
- The journaling file system
  - Write a **log entry** listing the three actions to be completed, and write the log entry to disk
  - Only after the log entry has been written to disk, the various operations can begin
    - After the operations complete successfully, the log entry is erased
    - If the system crashes, upon recovery the file system can check the log to see if any operations were pending.

Virtual File Systems



Position of the virtual file system.

Disk space management

- Strategies for storing an n byte file:
  - Allocate n consecutive bytes of disk space - segment
  - Allocate a number [n/k] blocks of size k bytes each - paging
- Problem of contiguous allocation
  - If the file grows it will have to be moved on the disk, it is an expensive operation and causes external fragmentation. =>
- Almost all file systems chop files up into fixed-size blocks that need not to be adjacent.

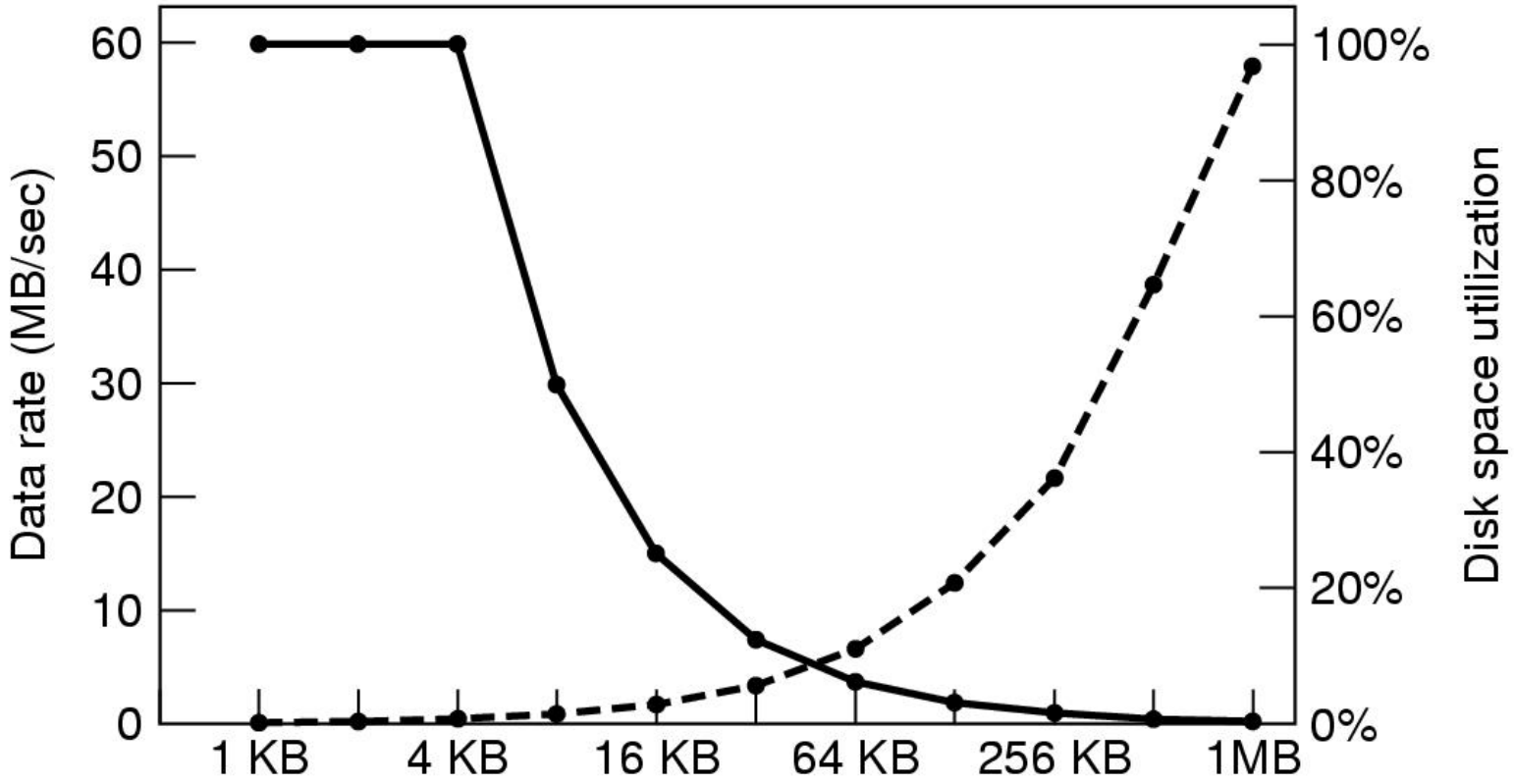
Block size (1)

- Review
  - File systems define a block size
    - Block size = 2^n * sector size
    - Contiguous sectors are allocated to a block
  - File systems view the disk as an array of blocks
    - Must allocate blocks to file
    - Must manage free space on disk

Block size (2)

- Large block sizes
  - Internal fragmentation
    - Last block has (on average) 1/2 wasted space
  - Lots of very small files; waste is greater.
  - Lower disk space utilization, but better performance
- Small block sizes
  - More seeks; file access will be slower.
  - Better disk utilization, but poor performance
- Usual size k = 1k, 2k or 4k

Block size (3)

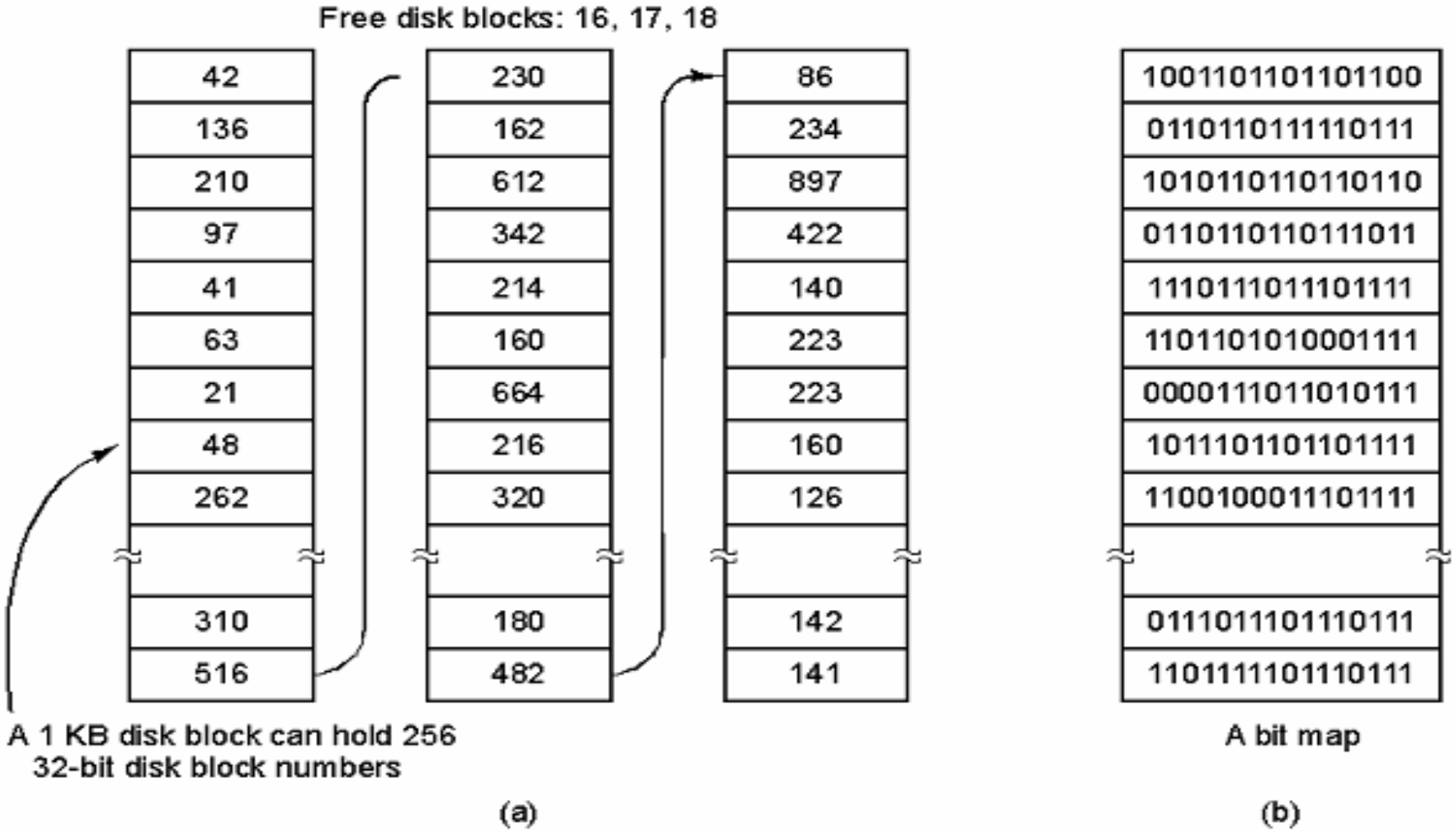


- Dotted line (left hand scale) gives data rate of a disk
- Dark line (right hand scale) gives disk space efficiency
- Assume all files are 4KB

Keeping Track of Free Blocks (1)

- Use linked list of disk blocks: each block holds as many free disk block numbers as will fit.
  - With 1 KB block and 32-bit disk block number ➔ 1024 * 8/32 = 256 disk block numbers ➔ 255 free blocks (and) 1 next block pointer.
- Use bit-map: A disk with n blocks requires a bit map with n bits
  - Free blocks are represented by 1's
  - Allocated blocks represented by 0's
  - 16GB disk has 2^{24} 1-KB and requires 2^{24} bits ➔ 2048 blocks
  - Using a linked list = $2^{24}/255$ = 65793 blocks. However, these blocks can be freed up as the disk is filled up.
- Bit map generally better if it can be kept completely in memory.

Keeping Track of Free Blocks (2)



(a) Storing the free list on a linked list
(b) A bit map

File System Reliability (1)

- The loss of a file system can be catastrophic.
- Backups are made to handle
  - Recover from disaster
  - Recover from stupidity (e.g. Recycle bin)
- Where to backup? Tertiary storage
  - Tape: holds 10 or 100s of GBs, costs pennies/GB
    - sequential access ⇒ high random access time
- Backup takes time and space

File System Reliability (2)

- Backup Issues
  - Should the entire FS be backup up?
    - Binaries, special I/O files usually not backed up
  - Do not backup unmodified files since last backup
    - Incremental dumps: complete per month, modified files daily
  - Compress data before writing to tape
  - How to backup an active FS?
    - Not acceptable to take system offline during backup hours
  - Security of backup media

File System Reliability (3)

- Two strategies for dumping a disk to tape:
  - **Physical dump**: starts at block 0 to the last one, write each block in order
    - Advantages: simple and fast
    - Disadvantages
      - Can not skip selected directories
      - Can not make incremental dumps
      - Can not restore individual files upon request

File System Reliability (4)

- Two strategies for dumping a disk to tape:
  - **Logical dump:** starts at one or more specified directories and recursively dumps all files and directories found that have changed since some given base date
    - Base date could be of last incremental dump, last full dump, etc.
    - Also dump all directories (even unmodified) that lie on the path to a modified file or directory

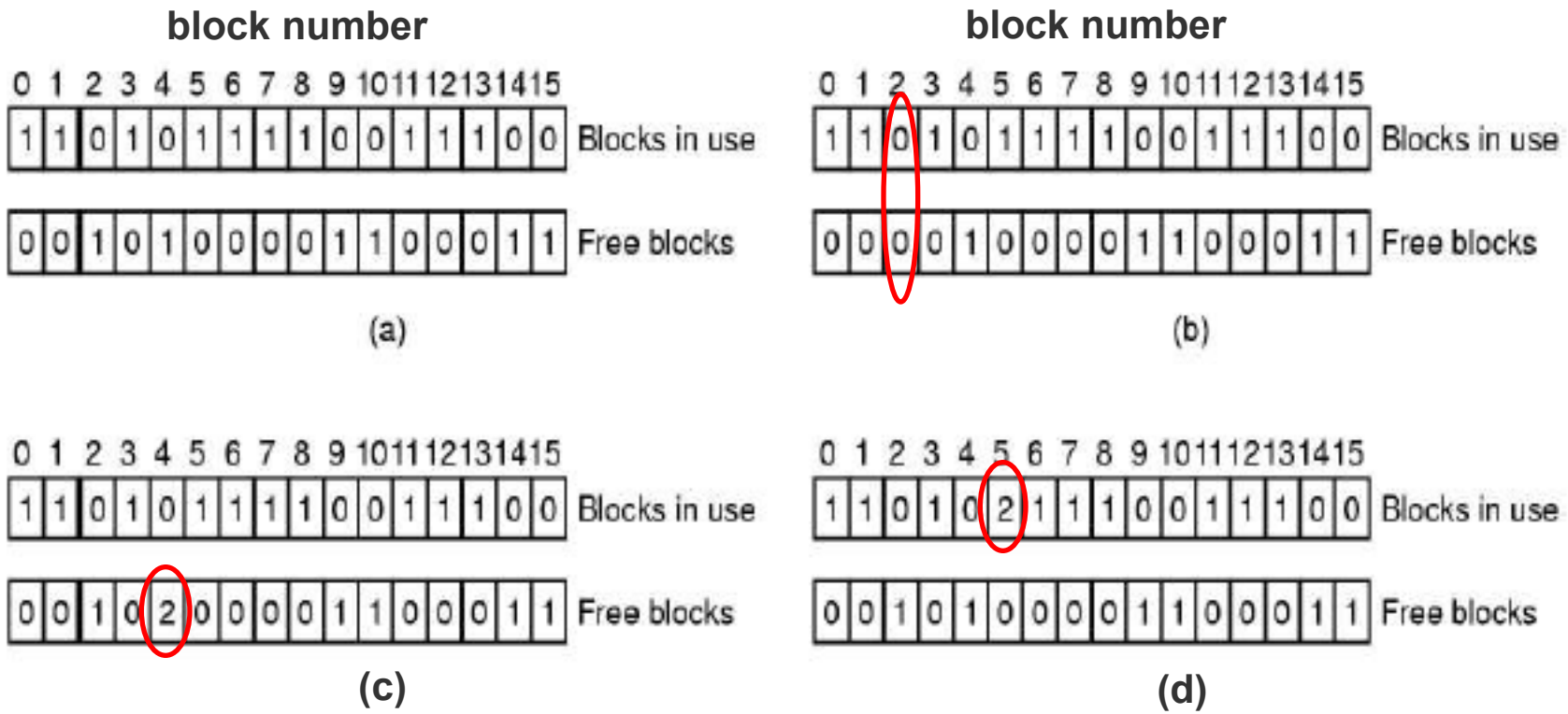
File System Reliability (5)

- File System Consistency
  - Most systems have a utility program that checks file system consistency
    - Windows: scandisk
    - UNIX: fsck
  - Two types of consistency checks can be made:
    - Blocks
    - Files (directory)
  - fsck: checking blocks
    - Reads all the i-nodes and mark used blocks
    - Examines the free list or bit maps and mark free blocks

File System Reliability (6)

- File System Consistency
  - Inconsistent States
    - Some block is not in a file or on free list (“missing block”)
      - Add it to the free list
    - Some block is on the free list more than once
      - Can’t happen when using a bitmap for free blocks
      - Fix the free list so the block appears only once
    - Some block is in more than one file
      - Allocate another block
      - Copy the block
      - Put it in a file
      - Notify the user that one file may contain data from another file
    - Some block is on free list and is in some file
      - Remove it from the free list

File System Reliability (7)



- File system states
  - (a) consistent
  - (b) missing block
  - (c) duplicate block in free list
  - (d) duplicate data block

File System Reliability (8)

- File System Consistency
  - fsck: checking directories
    - Use a per-file counter instead of per-block
    - Recursively descends the tree from the root directory, increasing the counter for each file, include hard links.
    - Compare these numbers with the link counts stored in the i nodes.
    - Problems: Reference count is too large or too small
    - Solution
      - Correct the reference count, force the link count in the i-node to the actual number of directory entries.

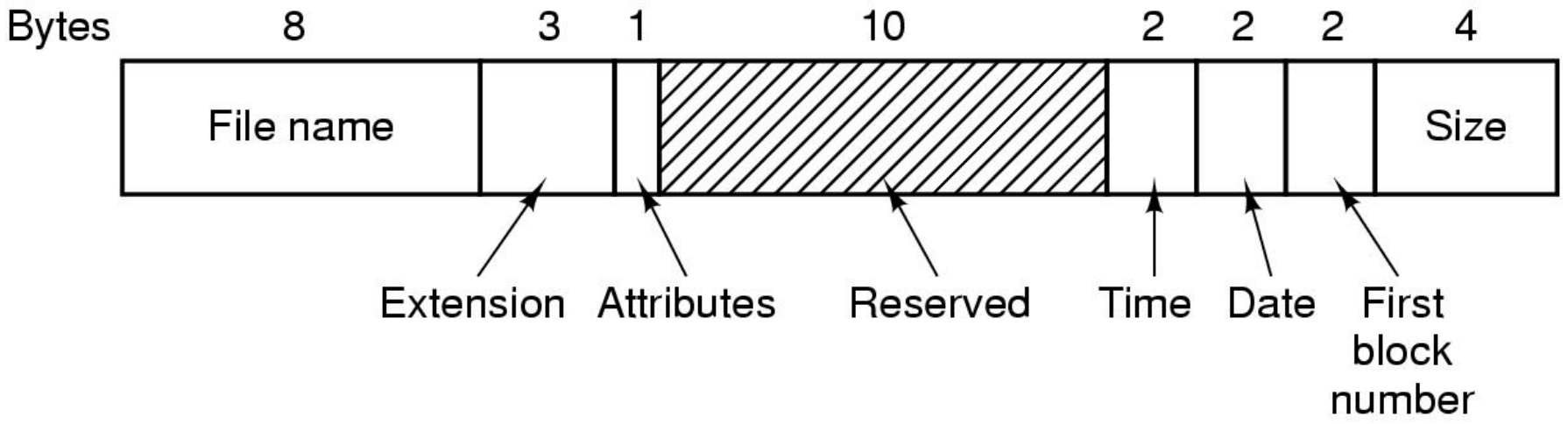
File System Performance

- Caching
  - Avoid disk I/O overhead
- Block Read Ahead
- Reducing Disk Arm Motion
  - Avoid seek overhead

Example File Systems

- The MS-DOS File System
- The Windows 98 File System
- The Unix V7 File System

The MS-DOS File System (1)



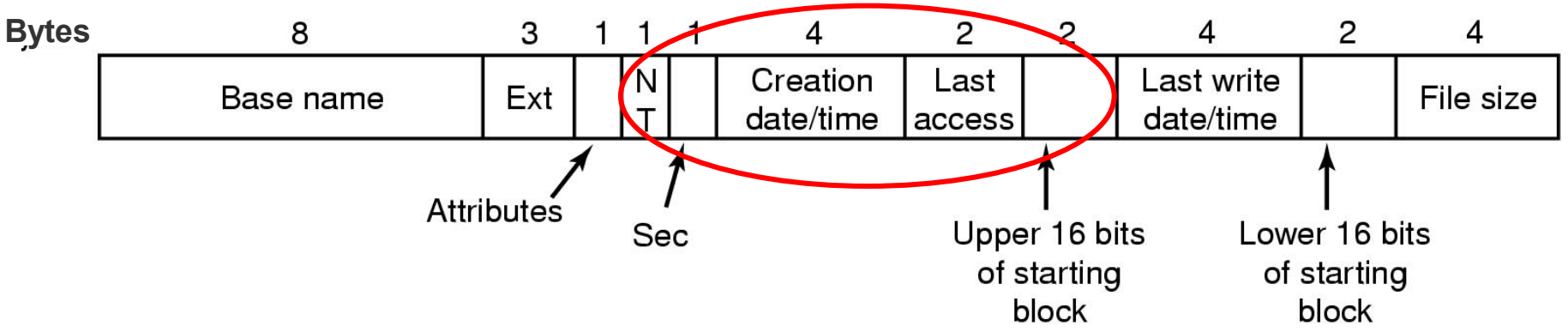
The MS-DOS directory entry

The MS-DOS File System (2)

Block size	FAT-12	FAT-16	FAT-32
0.5 KB	2 MB		
1 KB	4 MB		
2 KB	8 MB	128 MB	
4 KB	16 MB	256 MB	1 TB
8 KB		512 MB	2 TB
16 KB		1024 MB	2 TB
32 KB		2048 MB	2 TB

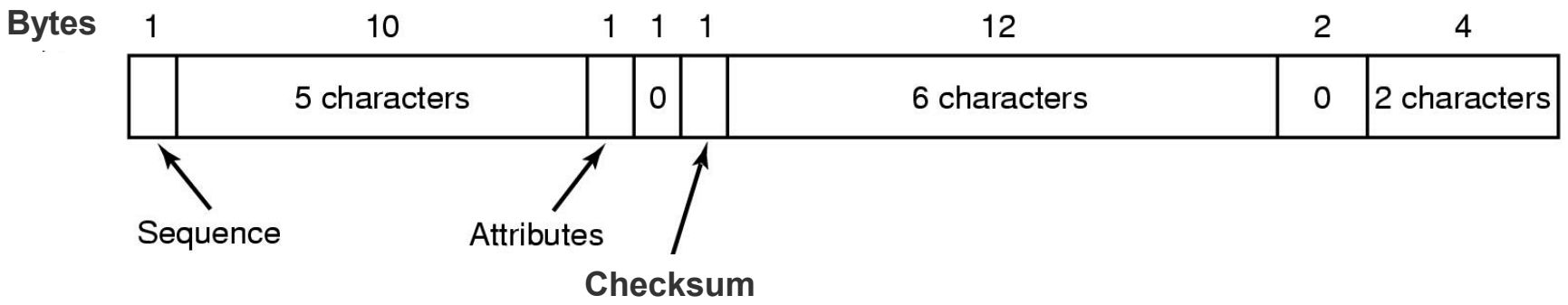
- Maximum partition for different block sizes
  (The empty boxes represent forbidden combinations)

The Windows 98 File System (1)



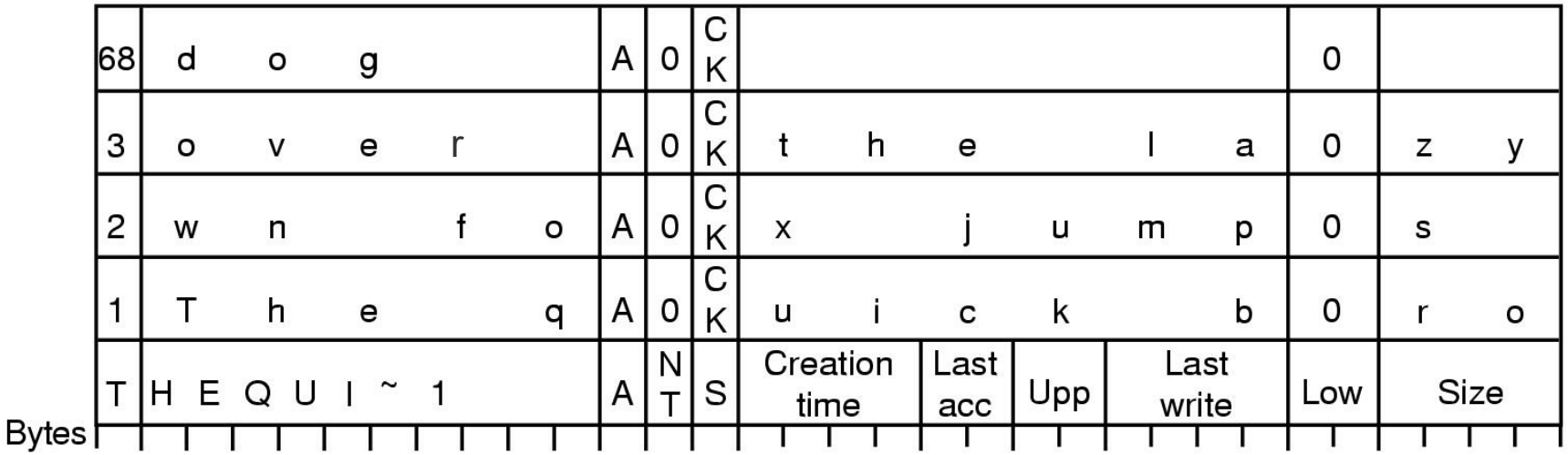
The extended MOS-DOS directory entry used in Windows 98

The Windows 98 File System (2)



An entry for (part of) a long file name in Windows 98

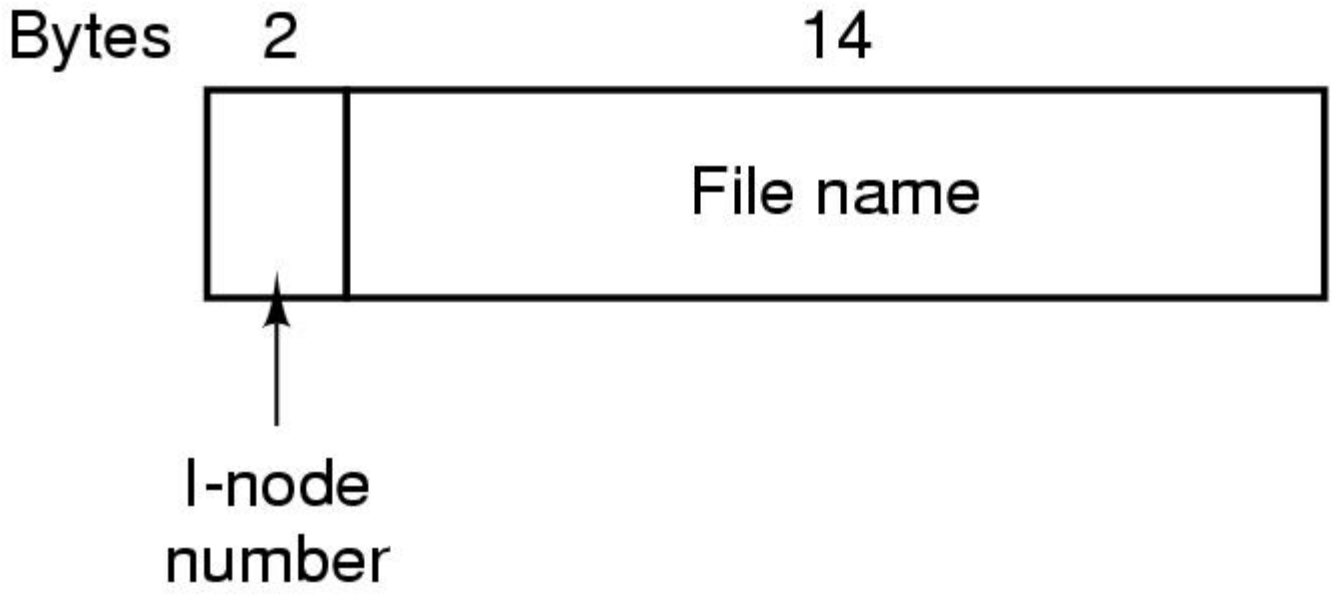
The Windows 98 File System (3)



An example of how a long name is stored in Windows 98

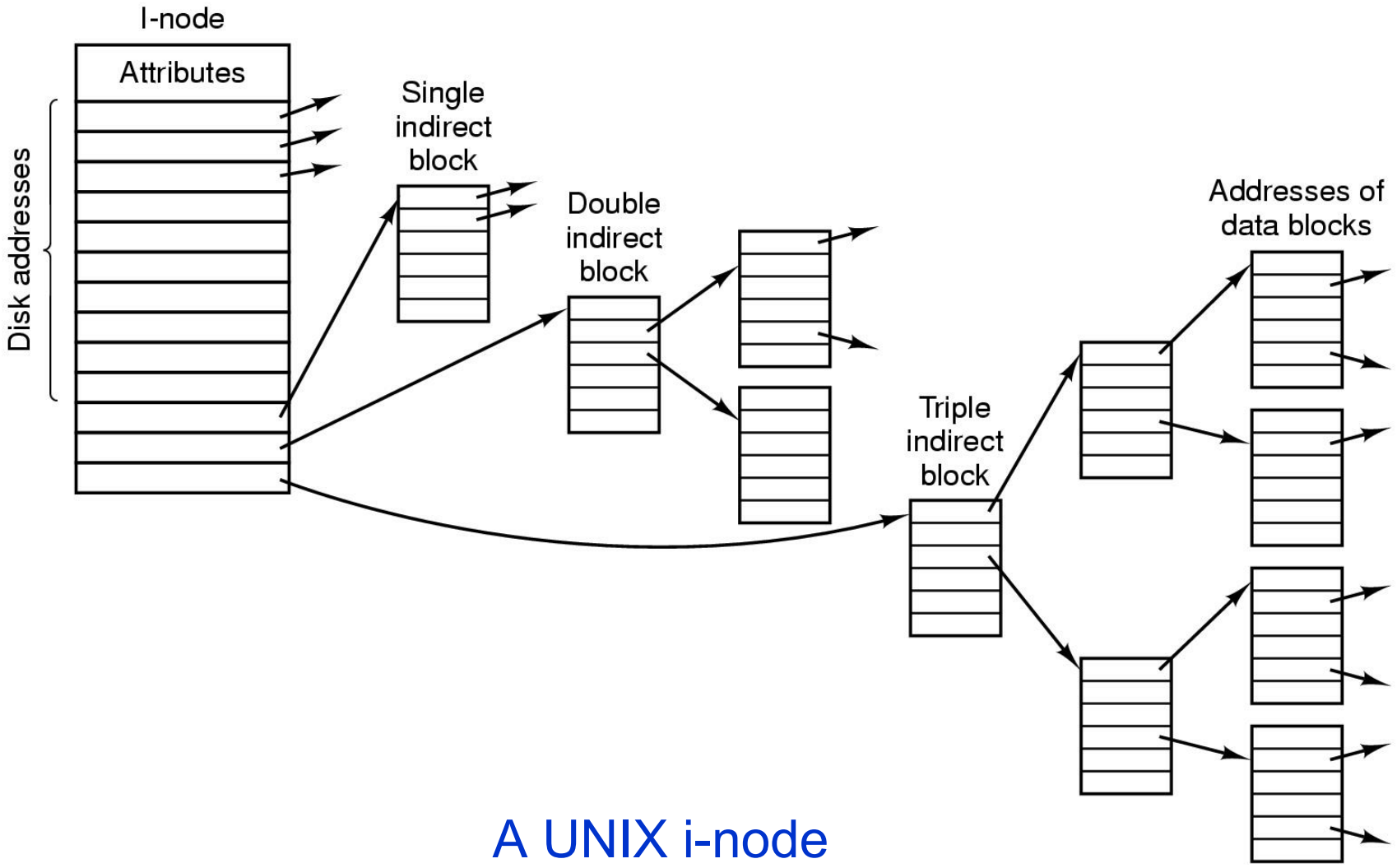
(File name: The quick brown fox jumps over the lazy dog)

The UNIX V7 File System (1)



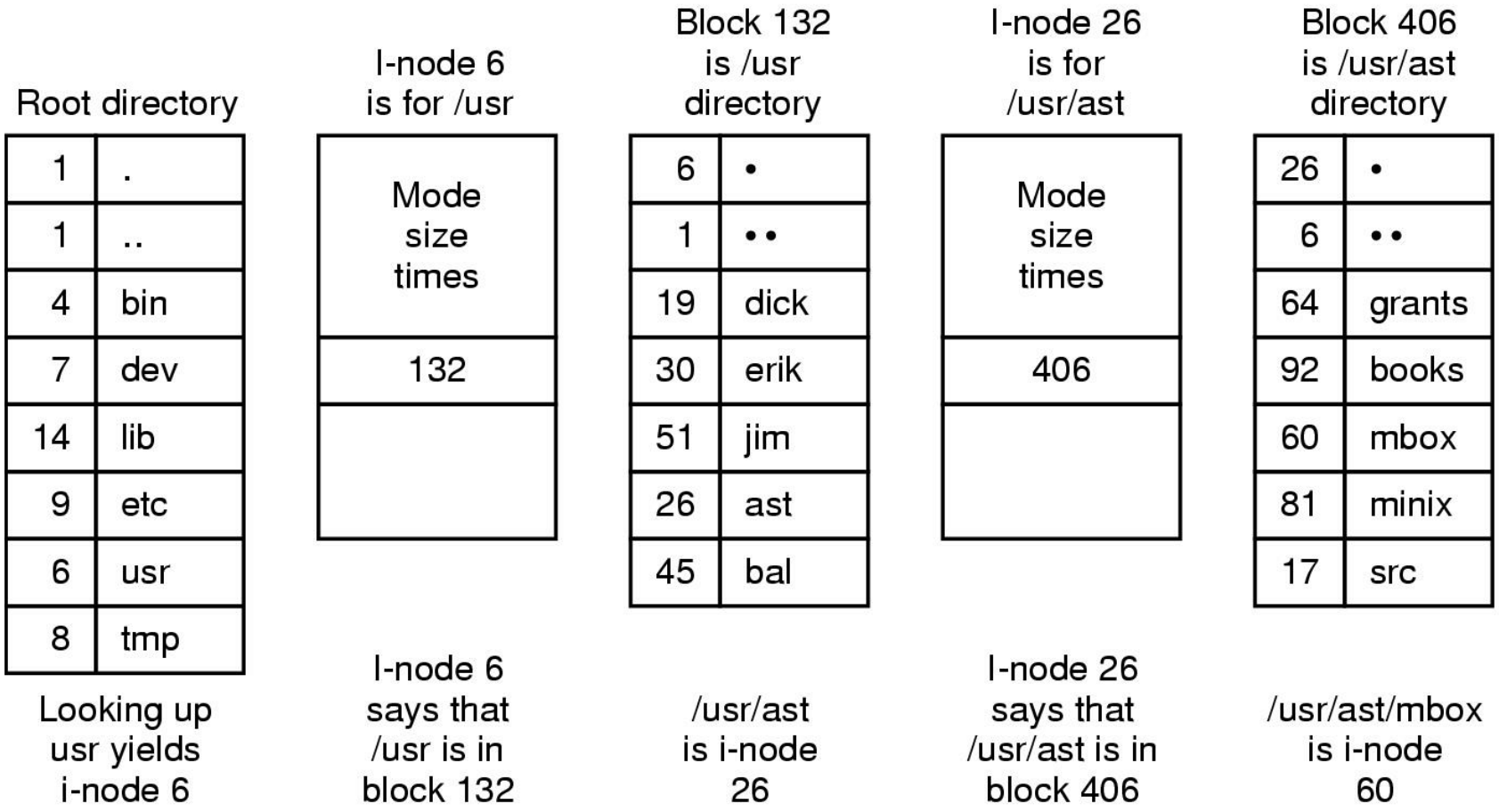
A UNIX V7 directory entry

The UNIX V7 File System (2)



A UNIX i-node

The UNIX V7 File System (3)



The steps in looking up */usr/ast/mbox*

Summary

- Files
- Directories
- File system implementation
- Example file systems